Meridian

Secure Job Search & Professional Networking Platform

CSE 345/545 — Foundations of Computer Security
Course Project Report

Team Members	Roll Number
Aniket Gupta	2022073
Angadjeet Singh	2022071
Harsh Kumar	2022198

IIIT Delhi

February 2026

Contents

1 Introduction

The modern job market relies heavily on digital platforms where sensitive personal data — resumes, credentials, professional histories — flows between job seekers, recruiters, and employers. Despite handling such critical information, many platforms treat security as an afterthought, leaving user data vulnerable to breaches, credential theft, and unauthorized access.

Meridian addresses this gap. Built for the CSE 345/545 *Foundations of Computer Security* course at IIIT Delhi, Meridian is a secure job search and professional networking platform where security is not bolted on but woven into every layer — from how passwords are stored to how resumes are encrypted on disk.

1.1 Design Philosophy

Our approach follows three guiding principles:

1. **Defense in depth** — No single layer is trusted. Authentication checks are layered with session validation, device fingerprinting, and CSRF verification. File uploads pass through extension, MIME, and magic-byte checks before reaching encryption.
2. **Secure by default** — Every endpoint requires authentication unless explicitly marked public. Passwords are hashed with Argon2id. Resumes are encrypted at rest with Fernet. Cookies are HttpOnly and Secure.
3. **Transparency through auditing** — Every security-relevant action is logged with context (IP, user agent, timestamp), creating a forensic trail for incident response.

1.2 Document Structure

This report walks through the project in a logical progression:

- **Section 2** defines what we set out to build — the full requirements specification with milestone mappings.
- **Section 3** explains how the system is structured — the technology choices, component layout, and data flow.
- **Sections 4 and 5** describe what we actually built, milestone by milestone, with implementation details and screenshots.
- **Section 6** zooms into the security mechanisms — threat models, attack mitigations, and access control.
- **Section 7** looks ahead to what's coming in Milestones 3 and 4.

2 Software Requirements Specification

This section maps the course-specified requirements to our implementation plan. Each requirement is tagged with its target milestone and current status, making it easy to track progress across the semester.

2.1 Authentication and Account Security

The platform must provide robust identity management. Users register with email and password, where passwords must meet strength requirements and are stored using a memory-hard hashing algorithm (never in plaintext). Login must support TOTP-based two-factor authentication, with session management via secure, HttpOnly cookies. Account lockout must trigger after repeated failed attempts.

ID	Requirement	Status
AS-1	Secure registration with password strength validation	M2 ✓
AS-2	Argon2id password hashing (64 MB, 3 iter)	M2 ✓
AS-3	TOTP two-factor authentication	M2 ✓
AS-4	JWT in HttpOnly, Secure, SameSite cookies	M2 ✓
AS-5	Device fingerprinting and session binding	M2 ✓
AS-6	Account lockout after 5 failed attempts	M2 ✓
AS-7	Refresh token rotation with reuse detection	M2 ✓

2.2 User Profiles

Users need to build a professional presence on the platform. This includes managing personal information, education history, work experience, and skills — all with input sanitization to prevent XSS attacks. Profile pictures undergo multi-layer validation (extension, MIME, magic bytes) before storage.

ID	Requirement	Status
UP-1	Create/edit profiles (name, headline, bio, location)	M2 ✓
UP-2	Avatar upload with magic-byte validation	M2 ✓
UP-3	Education and experience management (CRUD)	M2 ✓
UP-4	Skills with autocomplete from curated list	M2 ✓
UP-5	Connection requests and privacy controls	M3

2.3 Secure Resume Storage

Resumes are the most sensitive user asset on the platform. They must be encrypted at rest — meaning the server never stores plaintext resume content on disk. Downloads decrypt in memory and stream directly to the client. Access is restricted to the owner, authorized recruiters, and admins.

ID	Requirement	Status
SR-1	PDF upload with multi-layer validation	M2 ✓
SR-2	Fernet encryption at rest (AES-128-CBC + HMAC)	M2 ✓
SR-3	Owner/recruiter/admin-only access control	M2 ✓
SR-4	UUID filenames to prevent path traversal	M2 ✓

2.4 Admin Dashboard

Platform admins require visibility into user activity and the ability to moderate accounts. The dashboard must support user management (listing, role changes, suspension, deletion) and access to audit logs that record every security-relevant event.

ID	Requirement	Status
AD-1	Admin dashboard with user list and search	M2 ✓
AD-2	RBAC: User, Recruiter, Admin roles	M2 ✓
AD-3	Suspend/unsuspend accounts	M2 ✓
AD-4	Delete accounts with cascade cleanup	M2 ✓
AD-5	Audit log viewer with filters	M2 ✓

2.5 Upcoming Requirements (Milestones 3–4)

The following requirements are planned for future milestones:

- **M3:** Company pages, job postings, search with filters, application tracking, encrypted messaging
- **M4:** PKI integration (message signing, resume integrity), OTP virtual keyboard for high-risk actions, tamper-evident audit logs (hash chaining), defense demonstrations against SQL injection, XSS, CSRF, and session attacks

2.6 User Roles

The platform defines three roles with escalating privileges. Access control is enforced at the API layer via FastAPI's dependency injection — every protected endpoint passes through a role checker before any business logic executes.

1. **Regular User** — manages own profile, uploads resumes, applies to jobs
2. **Recruiter / Company Admin** — additionally creates company pages, posts jobs, views applicants
3. **Platform Admin** — full access to user management, moderation, and audit logs

3 System Architecture and Technology Stack

With the requirements defined, this section explains the architectural decisions that shape how Meridian is built and deployed.

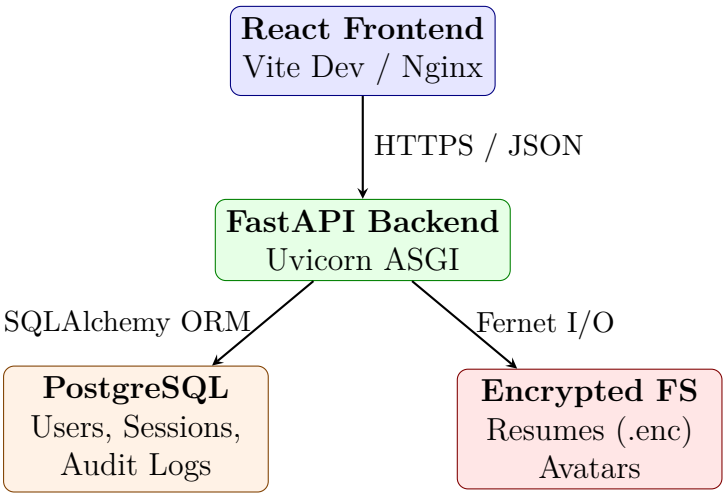
3.1 Technology Choices

Each technology was chosen to serve a specific security or development need:

- **FastAPI (Python)** — Provides native async support, automatic OpenAPI documentation, and — critically — Pydantic schema validation that rejects malformed input before it reaches business logic. Its dependency injection system cleanly separates authentication from route handlers.
- **React 18 + Vite** — Component-based architecture for reusable UI elements. JSX automatically escapes variables, providing built-in XSS protection on the client side.
- **PostgreSQL 16** — ACID-compliant with native UUID columns, JSONB for flexible audit log details, and robust access control. Parameterized queries via SQLAlchemy eliminate SQL injection.
- **Nginx** — Serves as a reverse proxy with TLS termination, adding security headers and rate limiting at the edge.
- **Docker + GitHub Actions** — Containerized builds ensure environment consistency. CI/CD pipelines automate linting, testing, and image publishing.

3.2 System Layout

The platform follows a three-tier architecture. The React frontend communicates with the FastAPI backend over HTTPS, which in turn interacts with PostgreSQL for structured data and an encrypted filesystem for resume storage.



3.3 Backend Organization

The backend follows a clean separation of concerns. Each module has a single responsibility, and dependencies flow downward:

- `routers/` — HTTP endpoint handlers, one file per feature (auth, users, resumes, admin, profile)
- `models/` — SQLAlchemy ORM models that define the database schema
- `schemas/` — Pydantic models that validate and sanitize all incoming data
- `security/` — Cryptographic primitives: password hashing, JWT issuance, TOTP, file encryption, CSRF tokens
- `dependencies.py` — Reusable authentication and RBAC checks injected via `Depends()`

This structure means a router never directly touches the database schema definition, and security primitives are never scattered across route handlers — they live in one auditable place.

3.4 Database Design

The database schema centers around the `users` table, with related tables for sessions, resumes, audit logs, and profile data:

Table	Purpose
`users`	Core accounts: Argon2id hash, Fernet-encrypted TOTP secret, role, suspension flag
`sessions`	JWT tracking: refresh token JTI, device fingerprint, revocation status, expiry
`resumes`	Metadata only (filename, path, size) — actual content is encrypted on disk
`audit_logs`	Immutable event log: action type, actor, IP, timestamp, JSONB detail payload
`used_otps`	TOTP replay prevention: stores used codes with their time-step window

3.5 CI/CD Pipeline

Every push to `main` triggers a two-stage pipeline:

1. **CI** — Runs `ruff` (Python linter) and `ESLint` (JS linter), then builds the frontend to verify no compilation errors.
2. **CD** — Builds Docker images for both backend and frontend, tags them with the commit SHA, and pushes to GitHub Container Registry.

This ensures that broken code never reaches the deployment stage, and every deployed image is traceable to a specific commit.

4 Milestone 1 — Foundation and Setup

4.1 Objectives

Milestone 1 focused on establishing the foundational infrastructure for the platform:

1. Finalize the technology stack
2. Configure HTTPS with TLS certificates
3. Deploy a skeleton application on the VM

4.2 Deliverables

4.2.1 Technology Stack Finalization

After evaluating multiple options, the following stack was selected:

- **Backend:** FastAPI (Python) — chosen for its native async support, automatic OpenAPI documentation, and Pydantic-based input validation with built-in type safety.
- **Frontend:** React 18 with Vite — chosen for component reusability, fast hot module replacement, and a rich ecosystem.
- **Database:** PostgreSQL 16 — chosen for ACID compliance, native UUID type, JSONB columns for flexible audit log details, and robust security features.
- **Deployment:** Docker + Nginx — containerized services with Nginx as reverse proxy and TLS terminator.

4.2.2 HTTPS Configuration

TLS was configured on the VM using:

- Self-signed certificates generated with OpenSSL for development
- Nginx configured as a reverse proxy with TLS termination
- `Strict-Transport-Security` header enabled
- All HTTP traffic redirected to HTTPS

4.2.3 Skeleton Application

A working skeleton was deployed with:

- FastAPI backend with health check endpoint (`/api/health`)
- React frontend with landing page and routing
- PostgreSQL database with initial schema
- Docker Compose configuration for all services
- `.env.example` with all required environment variables documented

5 Milestone 2 — Authentication, Profiles, Resumes, Admin

This milestone transforms the skeleton from M1 into a functional, security-hardened platform. It delivers five interconnected goals: secure authentication, TOTP-based 2FA, profile management, encrypted resume upload, and an admin dashboard. Each goal builds on the previous — authentication protects profiles, profiles hold resumes, and the admin dashboard oversees everything.

5.1 Goal 1: Secure Authentication

Authentication is the front door to every other feature, so we invested heavily in making it resilient.

5.1.1 Registration

When a user registers, their password goes through **Argon2id** hashing — a memory-hard algorithm designed to resist GPU-based cracking. We configure it with 64 MB memory cost, 3 iterations, and 4 parallel threads. The system also validates password strength (minimum 8 characters, mixed case, digits, and symbols) and rejects common passwords before hashing.

To prevent email enumeration, failed registrations return a generic error regardless of whether the email already exists.

5.1.2 Login

The login flow incorporates several defenses that work together:

1. **CSRF check** — The double-submit cookie pattern verifies the request is genuine.
2. **Lockout check** — After 5 failed attempts, the account locks for 15 minutes.
3. **Timing-safe verification** — If the email doesn't exist, we still compute a dummy Argon2 hash to prevent timing-based enumeration.
4. **2FA branch** — If TOTP is enabled, a short-lived temp token (5 min) is returned instead of session cookies. The user must verify with their authenticator app before gaining access.

On successful login, a database session is created with a device fingerprint (hashed IP + User-Agent) bound to the token. This means a stolen JWT cannot be used from a different device.

5.1.3 Token Architecture

We use a dual-token system stored exclusively in HttpOnly cookies — never in localStorage:

- **Access token** (15 min TTL) — Carries user ID, role, session ID, and device fingerprint. Used for API authorization.

- **Refresh token** (24h TTL) — Scoped to `/api/auth/refresh`. Contains a unique JTI that rotates on every refresh.

If a previously-used JTI appears (suggesting token theft), **all of that user's sessions are immediately revoked** — a defense inspired by OAuth 2.0 refresh token rotation best practices.

5.2 Goal 2: TOTP Two-Factor Authentication

Per instructor guidance, we implement TOTP (RFC 6238) rather than email/SMS OTP. This has practical advantages: it works offline, is compatible with standard authenticator apps (Google Authenticator, Authy), and requires no third-party SMS provider.

5.2.1 Setup

The setup process is designed to prevent accidental lock-outs:

1. The backend generates a random Base32 secret using `pyotp`.
2. The secret is **Fernet-encrypted** before being stored in the database — even a database breach won't expose TOTP seeds.
3. A QR code is generated from the `otpauth://` URI and sent to the frontend as Base64-encoded PNG.
4. The user scans the QR code and must *confirm* by entering a valid code.
5. Only after confirmation is 2FA activated. This prevents the user from enabling 2FA without actually setting up their authenticator.

5.2.2 Replay Prevention

A valid TOTP code is reusable within its 30-second window, which opens a replay attack vector. We close it by recording every verified code in a `used_otps` table with its time step. Before accepting a code, the system checks if it's been used in the current or adjacent windows (to account for clock drift). Old entries are garbage-collected after 5 minutes.

5.3 Goal 3: User Profile Management

With authentication in place, users can build their professional profiles.

5.3.1 Profile Data

The profile supports rich professional information: name, headline, location, bio, education history, work experience, and skills. Education and experience entries support full CRUD operations, while skills use an autocomplete interface backed by a curated list of 150+ professional skills.

5.3.2 Input Sanitization

Every text field passes through a sanitizer that strips HTML tags (via `bleach`), removes JavaScript event handlers, and blocks `javascript:` URIs. This server-side sanitization, combined with React's automatic JSX escaping on the client, creates a two-layer XSS defense.

5.3.3 Avatar Security

Profile pictures go through a validation pipeline that goes beyond simple extension checking:

1. Extension whitelist (`.jpg`, `.jpeg`, `.png`)
2. MIME type verification
3. **Magic byte validation** — the first bytes must match JPEG (`FF D8 FF`) or PNG (`89 50 4E 47`) signatures, preventing disguised file attacks
4. 2 MB size cap
5. UUID-based filename — the original name never reaches the filesystem

5.4 Goal 4: Encrypted Resume Storage

Resumes are the most sensitive data on the platform. A breach that exposes plaintext resumes would compromise names, addresses, phone numbers, and career histories of every user. Our approach ensures that even if an attacker gains filesystem access, the data remains encrypted.

5.4.1 The Upload Pipeline

Uploads go through a multi-stage process before reaching disk:

1. **Validation** — Extension (`.pdf`), MIME type (`application/pdf`), magic bytes (`%PDF-`), size (2 MB limit), and content scan for suspicious embedded JavaScript, `/Launch`, and `/OpenAction` commands.
2. **Encryption** — The validated content is encrypted with **Fernet** (AES-128-CBC + HMAC-SHA256). The encryption key is a 256-bit key stored in an environment variable, never in code.
3. **Storage** — Written as `<uuid>.enc` on disk. The original filename exists only in the database.

5.4.2 The Download Pipeline

On download, the process reverses — but critically, the decrypted content **never touches disk**. It's decrypted in memory and streamed directly to the client over HTTPS. Every download is recorded in the audit log.

5.4.3 Verification

Encrypted files on disk are completely unreadable without the key. Examining a stored resume shows only the Fernet token (starting with `gAAA...`), confirming that no plaintext leaks to the filesystem.

5.5 Goal 5: Admin Dashboard

The admin dashboard provides platform-wide visibility and control. Admins can list all users, change roles, suspend accounts (which immediately revokes all active sessions), and permanently delete users (cascading to resumes, sessions, and OTP records).

All admin actions — especially destructive ones like deletion — are logged in the audit trail with the admin's identity and the target user's details. This creates accountability even for privileged users.

5.5.1 Audit Events

The system logs over 20 distinct event types, including: registration, login success/failure, 2FA setup, account lockout, resume upload/download/deletion, role changes, suspensions, and — importantly — `refresh_token_reuse_detected`, which signals a potential token theft incident.

Each event captures the timestamp, actor ID, client IP, and a JSONB payload with contextual details specific to that event type.

5.6 User Interface

The frontend uses a dark theme with glassmorphism design and smooth animations. Below are screenshots of the key interfaces delivered in Milestone 2.

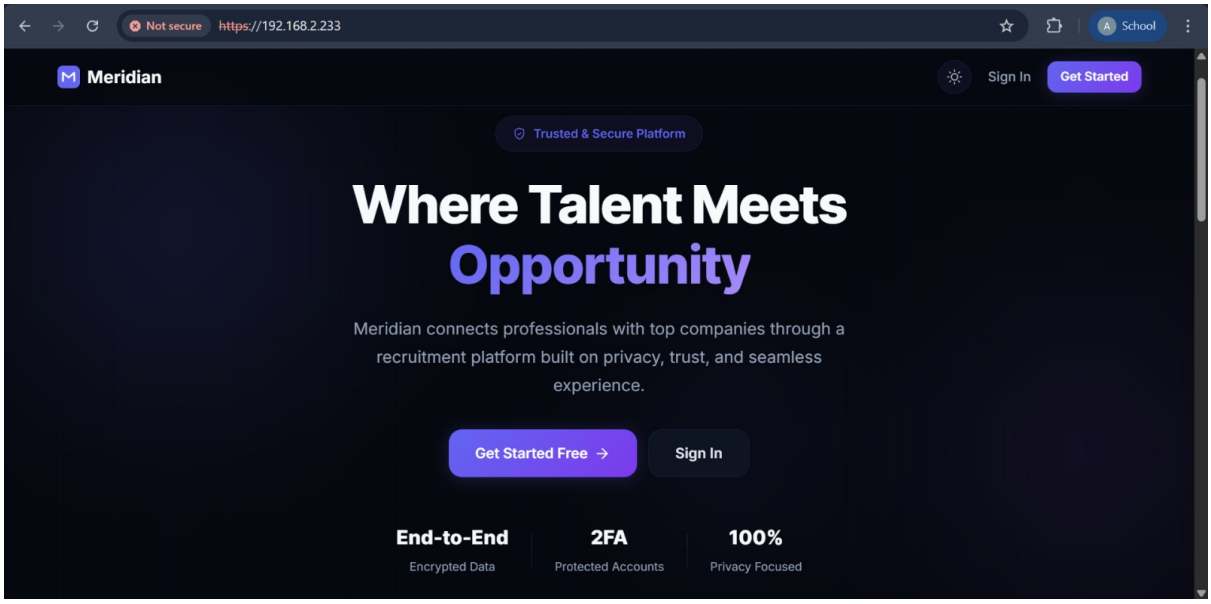


Figure 1: Landing page — hero section with value proposition and security highlights.

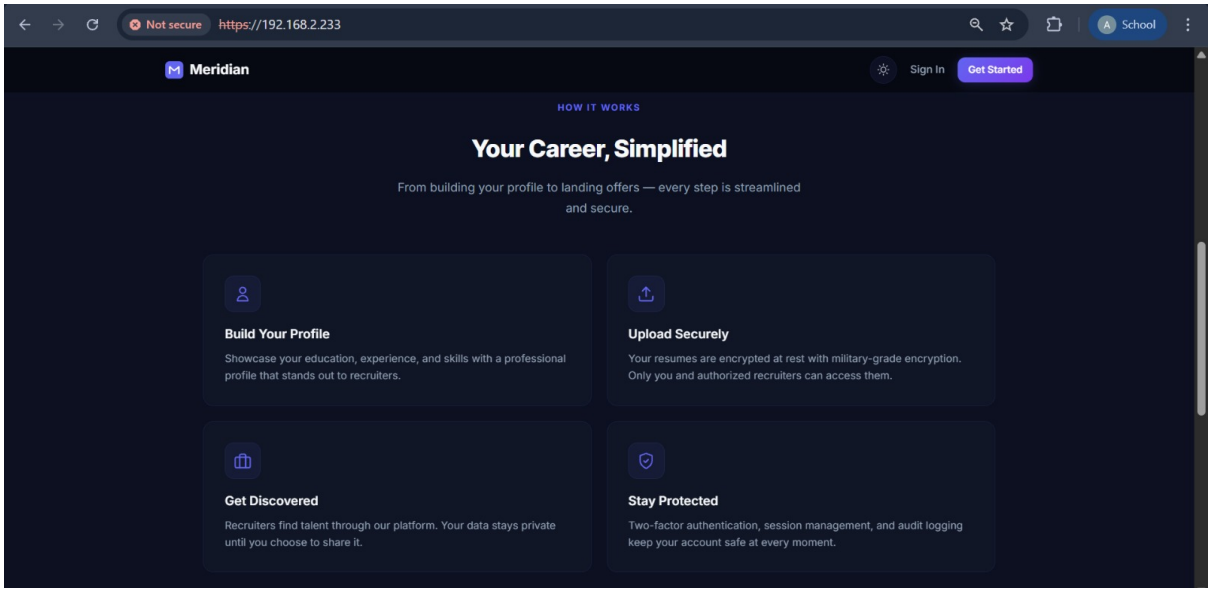
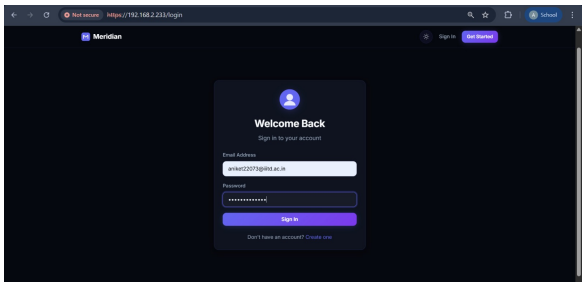
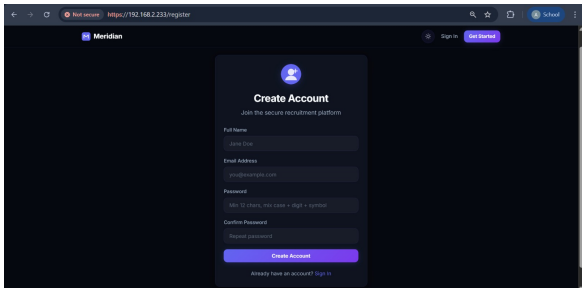


Figure 2: Landing page — feature cards highlighting encryption, 2FA, and audit capabilities.



(a) Login page



(b) Registration page

Figure 3: Authentication pages with gradient avatars and glassmorphism cards.

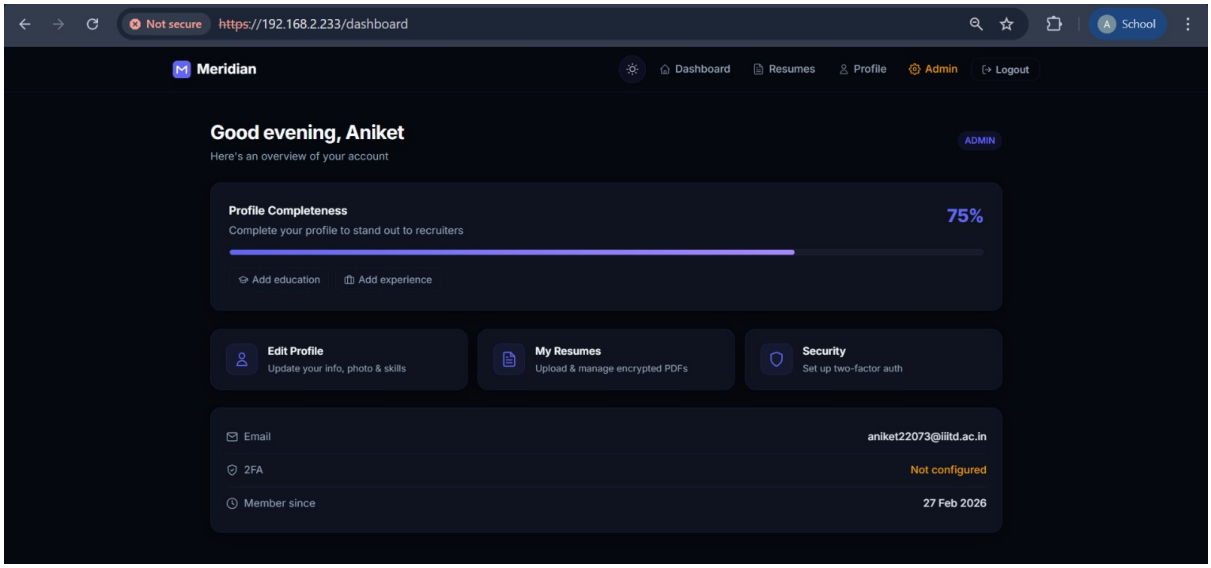


Figure 4: Dashboard — profile completeness indicator, quick actions, and account summary.

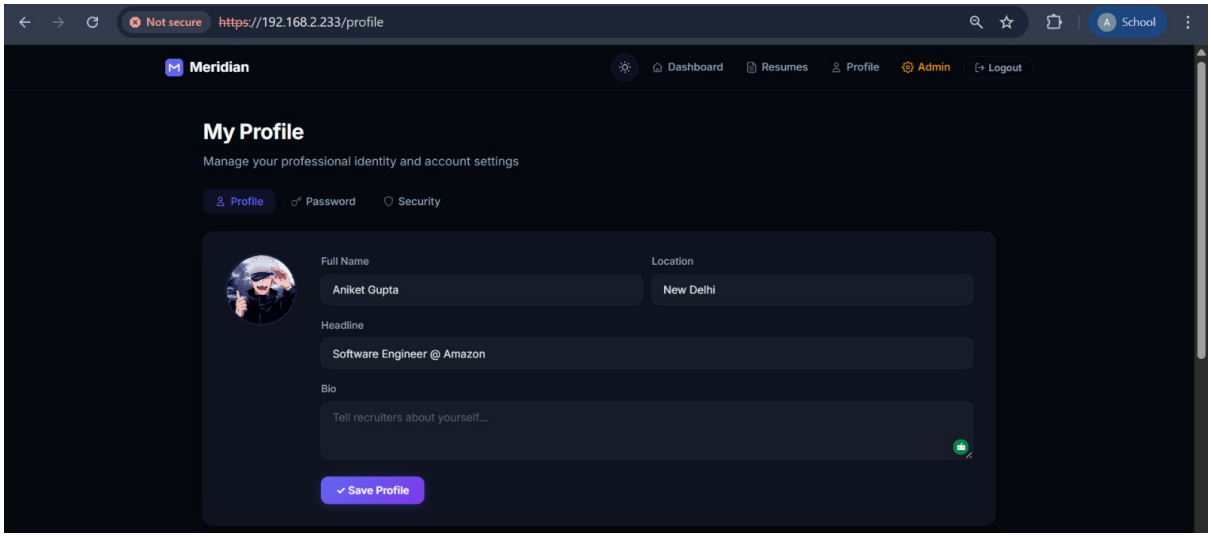


Figure 5: Profile management — education, experience, and skills with inline editing.

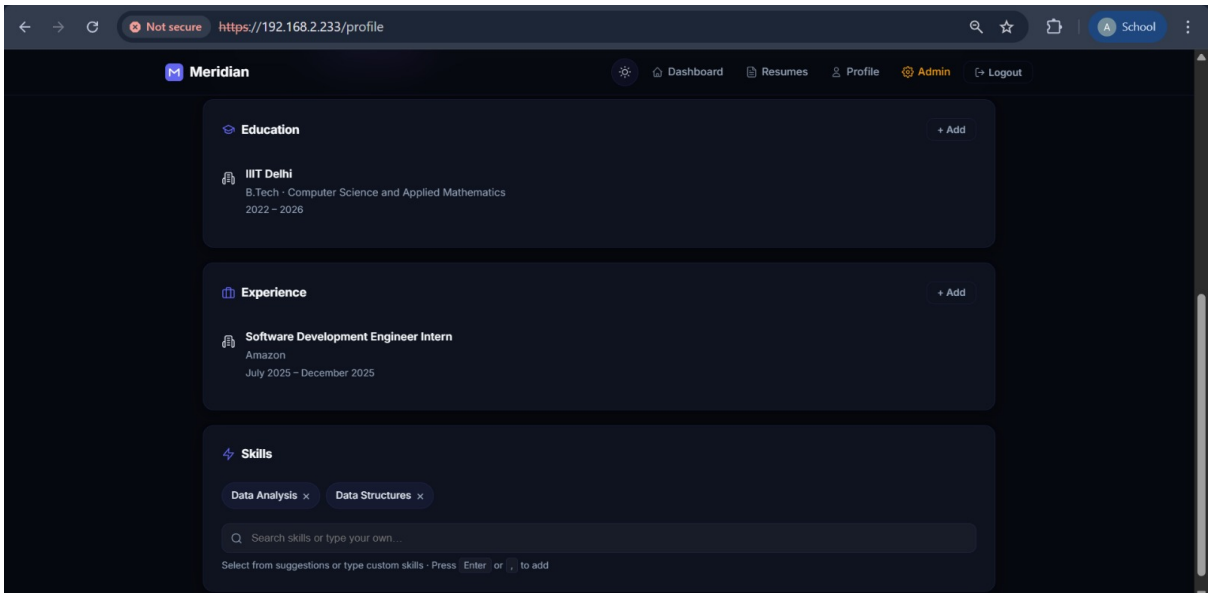


Figure 6: Profile fields — sanitized text inputs with real-time validation.

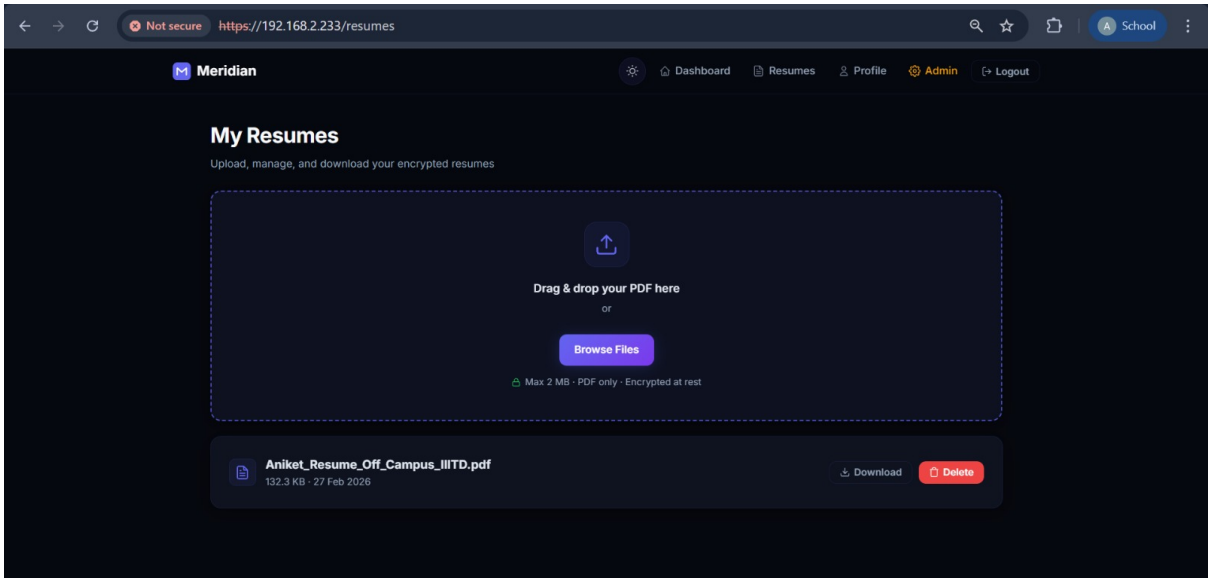


Figure 7: Resume management — encrypted upload, listing, and secure download.

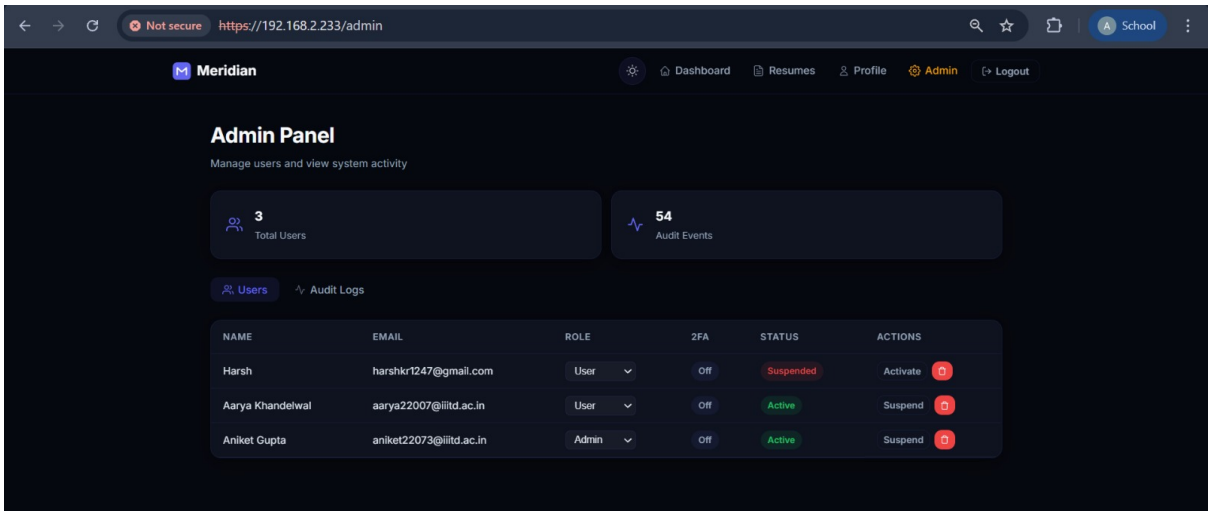


Figure 8: Admin panel — user management with role promotion, suspension, and deletion.

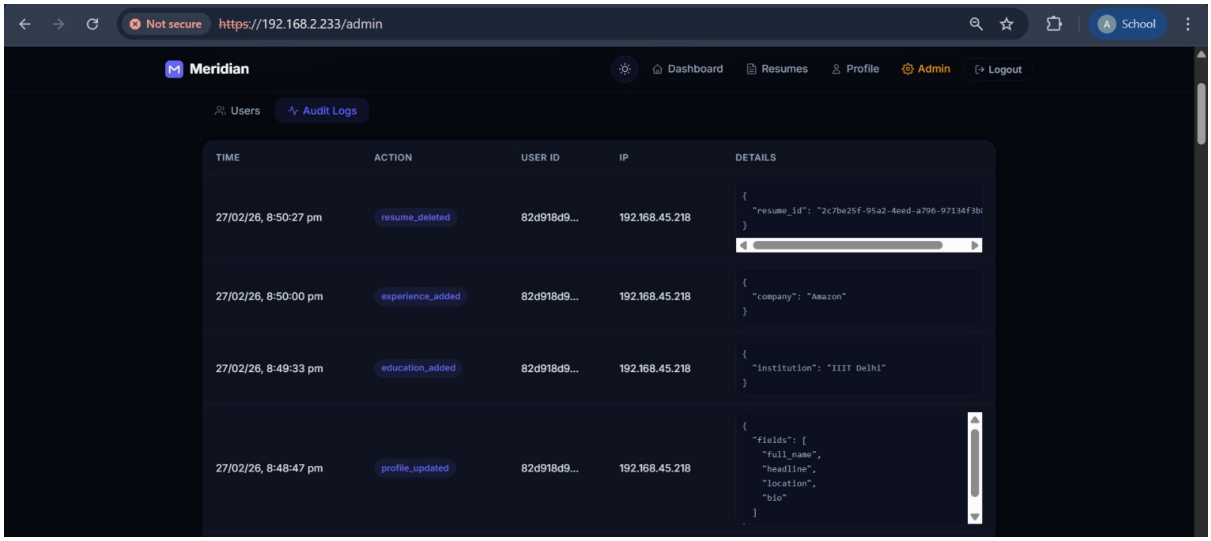


Figure 9: Audit logs — every security event with timestamp, actor, IP, and contextual details.

6 Security Analysis

This section takes a threat-driven look at the platform's security posture. Rather than listing features in isolation, we examine each attack vector and explain how the architecture defends against it.

6.1 Authentication Attacks

Password cracking. Even if the database is compromised, passwords are protected by Argon2id with 64 MB memory cost — making GPU-based brute force prohibitively expensive. Each hash uses a unique 16-byte salt.

Credential stuffing. Automated login attempts from breached credentials are throttled by account lockout (5 attempts triggers a 15-minute lock). The lockout event is logged, alerting admins to potential attacks.

User enumeration. Both registration and login return generic error messages. On login with an unknown email, a dummy hash is computed to make the response time indistinguishable from a valid attempt.

Token theft via XSS. JWTs live in HttpOnly cookies — JavaScript cannot access them, even in a successful XSS scenario. Combined with server-side input sanitization and React's automatic escaping, XSS attack surface is minimal.

Session hijacking. Each token is bound to a device fingerprint (hash of IP + User-Agent). A stolen token used from a different device will be rejected. Additionally, refresh token rotation means a stolen refresh token can only be used once before the entire session graph is invalidated.

6.2 Web Application Attacks

Cross-Site Scripting (XSS) is mitigated at two layers. Server-side, `bleach.clean()` strips all HTML tags, event handlers, and JavaScript URIs from user input before storage. Client-side, React's JSX escaping prevents DOM injection. Security headers (`X-Content-Type-Options: nosniff`) prevent MIME-sniffing attacks.

Cross-Site Request Forgery (CSRF) is prevented via the double-submit cookie pattern. A random token is set as both a cookie and returned in the response body. State-changing requests must include the token in an `X-CSRF-Token` header, which must match the cookie. Since an attacker's site cannot read our cookies (same-origin policy), they cannot forge the header.

SQL Injection is structurally prevented — all queries use SQLAlchemy's ORM with parameterized bindings. No raw SQL strings exist in the codebase. Input validation via Pydantic schemas adds a second layer of defense.

Session Fixation is impossible because sessions are created server-side with fresh UUIDs. The client cannot influence session identifiers. On every login, a new session is created; on logout, the existing session is database-revoked.

6.3 Data Protection

Sensitive data is protected both in transit and at rest:

- **Passwords** — Argon2id one-way hash (irreversible)

- **Resumes** — Fernet encryption on disk (AES-128-CBC + HMAC-SHA256); decrypted only in memory during authorized downloads
- **TOTP secrets** — Fernet-encrypted in the database; decrypted only during verification
- **Transport** — All traffic over HTTPS with TLS

File uploads face a five-layer validation pipeline (extension, MIME, magic bytes, size, content scan for PDFs) before encryption. Files are stored with UUID names to prevent path traversal, and the storage directory sits outside the web root.

6.4 Access Control

The RBAC model is enforced at the API layer, not the UI layer — meaning even if someone bypasses the frontend, the backend rejects unauthorized requests:

Resource	User	Recruiter	Admin
Own profile	Read/Write	Read/Write	Read/Write
Own resumes	CRUD	CRUD	CRUD
Others' resumes	—	Read	Read
User management	—	—	Full
Audit logs	—	—	Read

Table 1: RBAC permission matrix. Enforced via FastAPI `Depends()` injection.

6.5 Security Headers

Every response includes hardened headers that defend against common browser-based attacks:

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; script-src 'self';
   ...
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

7 Future Milestones

With Milestone 2 establishing the security foundation, the remaining milestones layer on collaborative features and advanced security mechanisms.

7.1 Milestone 3 — March 31, 2026

Milestone 3 shifts focus from individual user features to multi-user interactions:

- **Company pages and job postings** — Recruiters can create company profiles and publish job listings with titles, descriptions, required skills, location, and salary.
- **Job search** — Keyword search with filters for remote work, experience level, and skills.
- **Application tracking** — Candidates apply with their encrypted resumes. Application status flows through Applied → Reviewed → Interviewed → Offer/Rejected.
- **Encrypted messaging** — End-to-end encrypted 1:1 messaging between recruiters and candidates. The server stores only ciphertext.

7.2 Milestone 4 — Final Evaluation — April 30, 2026

The final milestone adds the most security-intensive features:

- **PKI integration** — At least two functions backed by public key infrastructure (e.g., message signing, resume integrity verification via digital signatures).
- **OTP with virtual keyboard** — A visual on-screen keyboard for TOTP entry during high-risk actions (password reset, account deletion), defending against keyloggers.
- **Tamper-evident audit logs** — Hash chaining where each log entry includes the hash of the previous entry, making retroactive tampering detectable.
- **Attack demonstrations** — Live demonstrations of defense against SQL injection, XSS, CSRF, session fixation, and session hijacking.

7.3 Bonus Targets

- **Blockchain audit logging** — Extending hash chaining into a lightweight blockchain structure for stronger tamper evidence.
- **Resume parsing** — NLP-based extraction of skills and experience from uploaded PDFs for automated candidate–job matching.